



FUNDAMENTOS Y TEORÍA DE LA COMPUTACIÓN

AUTÓMATAS FINITOS

Autómatas finitos:

Un autómata finito es una estructura matemática que representa un sistema o máquina abstracta.

Los autómatas finitos pueden considerarse como mecanismos aceptadores o reconocedores de palabras.

AUTÓMATAS FINITOS

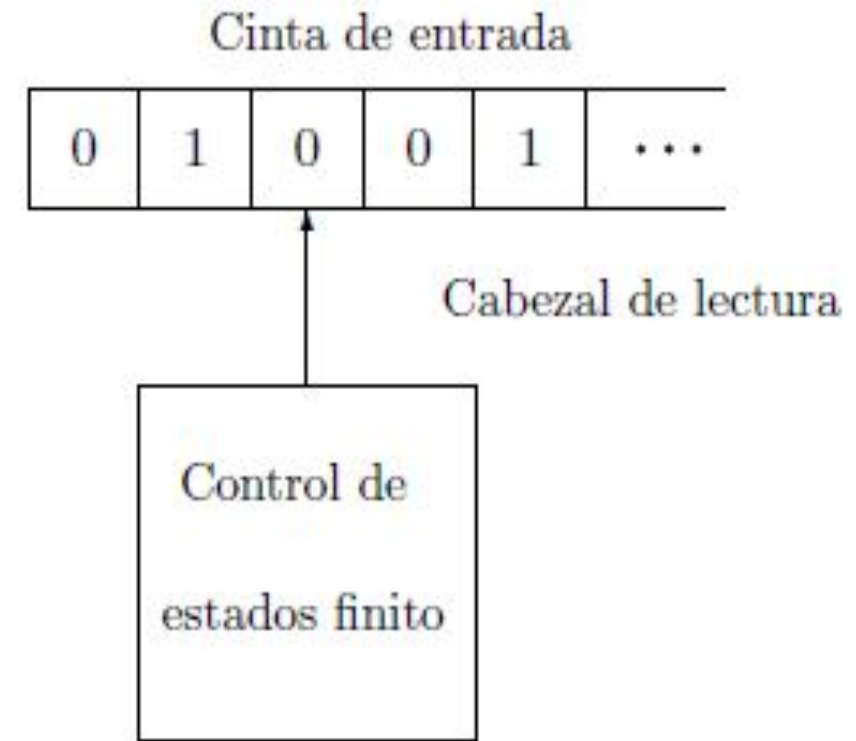
Arquitectura de un autómata finito (AF):

La cinta de entrada está dividida en celdas capaz de almacenar un símbolo de un alfabeto.

La máquina es capaz de leer los símbolos de izquierda a derecha por medio de un cabezal de lectura. Cada vez que se lee un símbolo, el cabezal de lectura se mueve a la siguiente celda a la derecha y la máquina efectúa un cambio de estado o transición.

La transición está determinada por el mecanismo de control (que contiene un número finito de estados), programado para conocer cual debe ser el nuevo estado, que dependerá de la combinación del estado actual y el símbolo de entrada leído.

De manera informal decimos que un autómata finito aceptará una palabra de entrada si, comenzando por un estado especial llamado estado inicial y estando la cabeza de lectura apuntando al primer símbolo de la cadena, la máquina alcanza un estado final o de aceptación después de leer el último símbolo de la cadena.



AUTÓMATAS FINITOS

Autómatas finitos deterministas:

Un autómata finito determinista (AFD) se define como una quintupla $M = (Q, V, \delta, q_0, F)$, donde:

- Q es un conjunto finito de estados
- V es el alfabeto de entrada
- q_0 es el estado inicial
- $F \subseteq Q$ es el conjunto de estados finales
- $\delta : Q \times V \rightarrow Q$ es la función de transición

El nombre “determinista” viene de la forma en que está definida la función de transición: si en un instante t la máquina está en el estado q y lee el símbolo a entonces, en el instante siguiente $t + 1$ la máquina cambia de estado y sabemos con seguridad cual es el estado al que cambia, que es precisamente $\delta(q, a)$.

AUTÓMATAS FINITOS

Función de transición de un AFD:

La función de transición de un AFD se puede representar de dos formas: mediante una tabla de transición o mediante un diagrama de transición.

Tabla de transición:

- Cada fila corresponde a un estado $q \in Q$
- El estado inicial se precede del símbolo $\rightarrow$
- Cada estado final se precede del símbolo $\#$
- Cada columna corresponde a un símbolo de entrada $a \in V$
- En la posición (q, a) está el estado que determine $\delta(q, a)$

Diagrama de transición:

- Los nodos se etiquetan con los estados
- El estado inicial tiene un arco entrante no etiquetado
- Los estados finales están rodeados de un doble círculo
- Habrá un arco etiquetado con a desde el nodo q_i al q_j si $\delta(q_i, a) = q_j$

AUTÓMATAS FINITOS

Ejemplo:

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

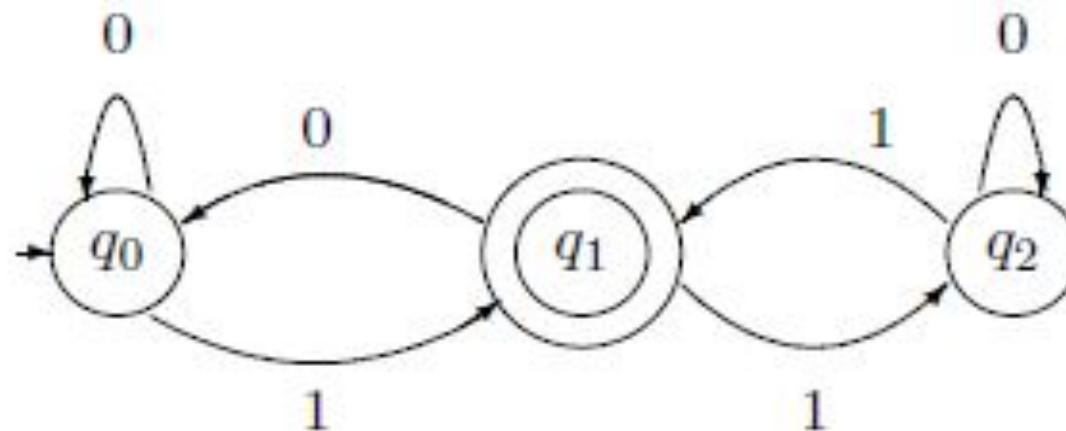
$$\delta : \{q_0, q_1, q_2\} \times \{0, 1\} \longrightarrow \{q_0, q_1, q_2\}$$

$$\delta(q_0, 0) = q_0 \quad \delta(q_0, 1) = q_1$$

$$\delta(q_1, 0) = q_0 \quad \delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_2 \quad \delta(q_2, 1) = q_1$$

δ	0	1
$\rightarrow q_0$	q_0	q_1
$\# q_1$	q_0	q_2
q_2	q_2	q_1



AUTÓMATAS FINITOS

Autómatas finitos no deterministas:

Un autómatata finito no determinista (AFND) es una quintupla $M = (Q, V, \Delta, q_0, F)$ donde todos los componentes son como en los AFDs, excepto la función de transición que se define ahora como:

$$\Delta: Q \times V \rightarrow P(Q)$$

donde $P(Q)$ denota el conjunto de las partes de Q (o conjunto potencia $2Q$).

Característica de “no determinismo”: a partir del estado actual y del símbolo actual de entrada no se puede determinar de forma exacta cuál será el estado siguiente. Por ejemplo, podemos que dado el estado actual q y el símbolo de entrada a , el estado siguiente puede ser cualquier estado entre q_1 y q_m . También puede darse el caso de que el estado siguiente no está definido.

Un AFND acepta una palabra de entrada w siempre que sea posible comenzar por el estado inicial y que exista una secuencia de transiciones que nos lleven a consumir la palabra y acabe el autómatata en un estado final. Pueden existir otras secuencias de transiciones que no acaben en estado final, pero basta que exista una que acabe en estado final para que la palabra sea aceptada.

AUTÓMATAS FINITOS

Los AFND también se representan mediante tablas o diagramas de transición. En el diagrama de transición, el no determinismo se descubre porque hay algún nodo del que parten dos o más arcos etiquetados con el mismo símbolo del alfabeto, o falta algún arco para algún símbolo del alfabeto. Por ejemplo:

δ	a	b
$\rightarrow q_0$	$\{q_0, q_3\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
$\# q_2$	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_4\}$	$\emptyset$
$\# q_4$	$\{q_4\}$	$\{q_4\}$

Tabla de transición

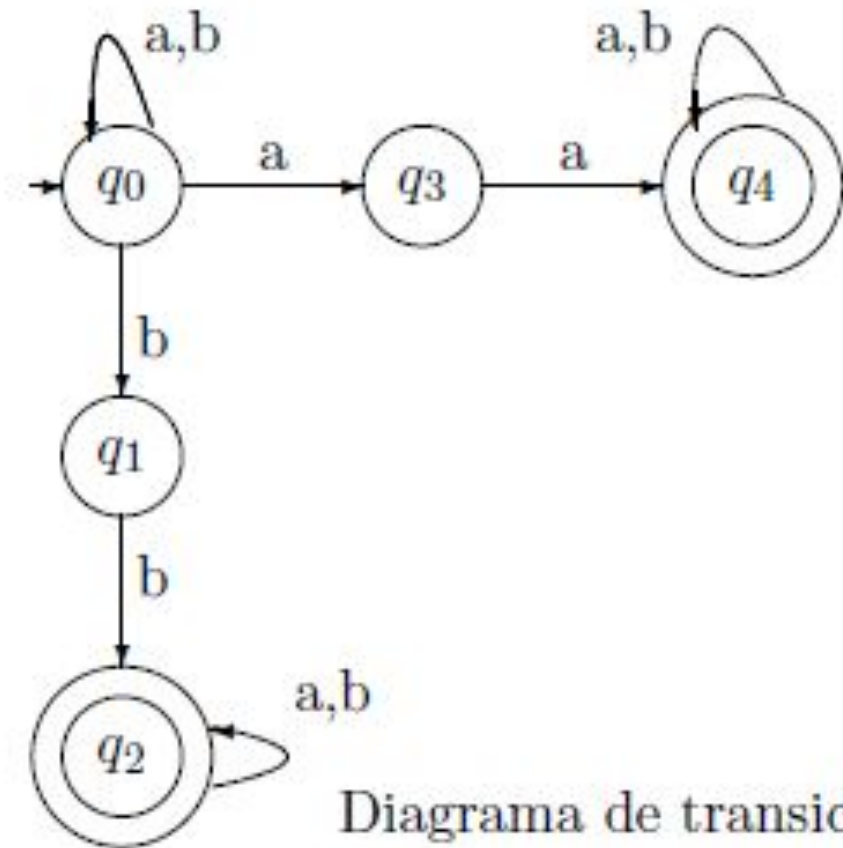


Diagrama de transición

AUTÓMATAS FINITOS

Autómatas finitos no deterministas con λ transiciones:

Un autómata finito no determinista con λ transiciones (AFND- λ) es una quintupla $M = (Q, V, \Delta, q_0, F)$ donde todos los componentes son como en los AFND con el agregado de las λ transiciones.

Una λ transición permite al autómata pasar de un estado a otro sin consumir símbolos de la entrada.

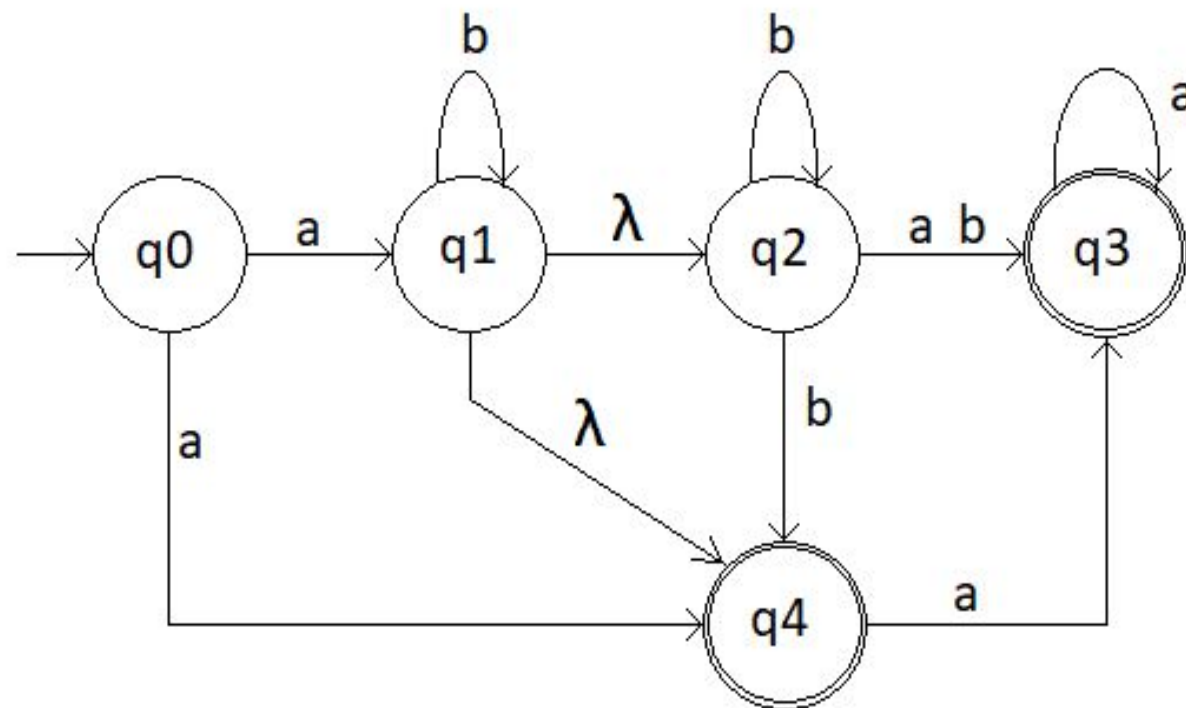
Las λ transiciones se representan en la tabla de transiciones agregando una columna que se corresponde con el símbolo λ e indica todas las λ transiciones del autómata.

En el diagrama del autómata, se representan con un arco etiquetado con el símbolo λ .

AUTÓMATAS FINITOS

Ejemplo:

Δ	a	b	λ
$\rightarrow q_0$	{q1, q4}	$\emptyset$	$\emptyset$
q1	$\emptyset$	{q1}	{q2, q4}
q2	{q3}	{q2, q3, q4}	$\emptyset$
# q3	{q3}	$\emptyset$	$\emptyset$
# q4	{q3}	$\emptyset$	$\emptyset$



AUTÓMATAS FINITOS

Configuración de un autómata finito:

La configuración de un autómata finito en cierto instante viene dada por el estado del autómata en ese instante y por la porción de cadena de entrada que le queda por leer o procesar.

La porción de cadena leída hasta llegar al estado actual no tiene influencia en el comportamiento futuro de la máquina. Un AF es una máquina sin memoria externa.

Formalmente una configuración de un AF es un elemento $(q, w) \in (Q \times V^*)$.

Algunos tipos de configuraciones especiales son:

- **Configuración inicial:** (q_0, w) , donde q_0 es el estado inicial y w la palabra de entrada.
- **Configuración de parada:** cualquier configuración en la que el autómata puede parar su ejecución, porque se haya procesado toda la entrada o porque se haya llegado a una situación donde no es aplicable ninguna transición.
- **Configuración de aceptación:** (q_F, λ) , donde q_F es un estado final del autómata. Una vez alcanzada esta configuración el autómata puede aceptar la palabra.

AUTÓMATAS FINITOS

Relación de cálculo en un paso:

Si dos configuraciones C_i y C_j están relacionadas mediante la relación $\vdash$ ($C_i \vdash C_j$), quiere decir que se puede pasar de la configuración C_i a la C_j aplicando una sola transición.

Formalmente:

AFD:

$$(q, w) \vdash (q', w') \Leftrightarrow \begin{cases} w = aw', \text{ donde } a \in V \\ q' = \delta(q, a) \end{cases}$$

AFND:

$$(q, w) \vdash (q', w') \Leftrightarrow \begin{cases} w = aw', \text{ donde } a \in V \\ q' \in \Delta(q, a) \end{cases}$$

AFND- λ :

$$(q, w) \vdash (q', w') \Leftrightarrow \begin{cases} w = \sigma w', \text{ donde } \sigma \in V \cup \{\lambda\} \\ q' \in \Delta(q, \sigma) \end{cases}$$

AUTÓMATAS FINITOS

Relación de cálculo:

Se dice que la configuración C_i alcanza la configuración C_j ($C_i \vdash^* C_j$), si se cumple una de las dos condiciones siguientes:

1. $C_i = C_j$, o bien,
2. $\exists C_0, C_1, \dots, C_n$, tal que $C_0 = C_i$, $C_n = C_j$, y $\forall 0 \leq k \leq n - 1$ se cumple que $C_k \vdash C_{k+1}$

Una secuencia del tipo $C_0 \vdash C_1 \vdash \dots \vdash C_n$ se llama cálculo en n pasos, abreviadamente $C_0 \vdash_{n \text{ pasos}}^* C_n$.

AUTÓMATAS FINITOS

Lenguaje aceptado por un AF:

Un autómata finito sirve para reconocer cierto tipo de lenguajes.

Para un autómata finito $M = (Q, V, \delta, q_0, F)$, se define el lenguaje aceptado por M , $L(M)$, como:

$$L(M) = \{w \in V^* \mid (q_0, w) \vdash^* (q_F, \lambda) \text{ donde } q_F \in F\}$$

Una palabra w será aceptada por el autómata M , si partiendo de la configuración inicial con w en la cinta de entrada, el autómata es capaz de alcanzar una configuración de aceptación.

En un autómata finito determinista, el hecho de que una palabra w sea aceptada por el autómata asegura que existe un único camino en el diagrama de transición que lleva del nodo etiquetado con el estado inicial al nodo etiquetado con el estado final y cada arco que se recorre en este camino está etiquetado con un símbolo de la palabra.

AUTÓMATAS FINITOS

Autómatas finitos y expresiones regulares:

Las expresiones regulares que pueden utilizarse para describir lenguajes regulares generados por gramáticas regulares o de tipo 3, según la clasificación de Chomsky.

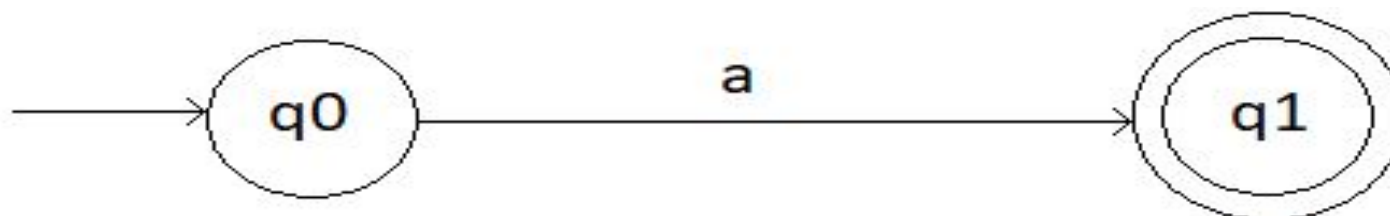
Si un lenguaje es aceptado por un autómata finito también se puede describir mediante una expresión regular y al contrario, todo lenguaje descrito por una expresión regular puede ser aceptado por un autómata finito.

Teorema de Análisis de Kleene: Si L es un lenguaje aceptado por un autómata finito M entonces existe una expresión regular α tal que $L = L(M) = L(\alpha)$.

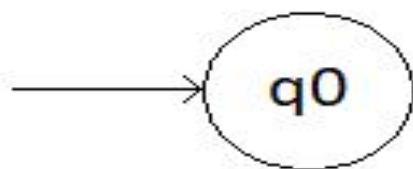
Teorema de Síntesis de Kleene: Si L es un lenguaje asociado a una expresión regular α entonces existe un autómata finito M tal que $L = L(\alpha) = L(M)$.

AUTÓMATAS FINITOS

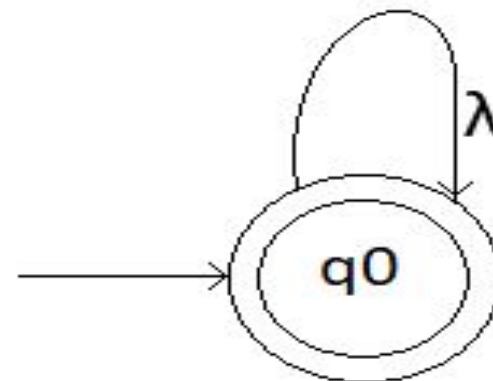
Autómatas finitos a partir de expresiones regulares simples:



AFN que acepta el lenguaje unitario $\{a\}$



AFN que acepta el lenguaje vacío $\emptyset$



AFN que acepta el lenguaje unitario $\{\lambda\}$

AUTÓMATAS FINITOS

Autómatas finitos para unión de expresiones regulares:

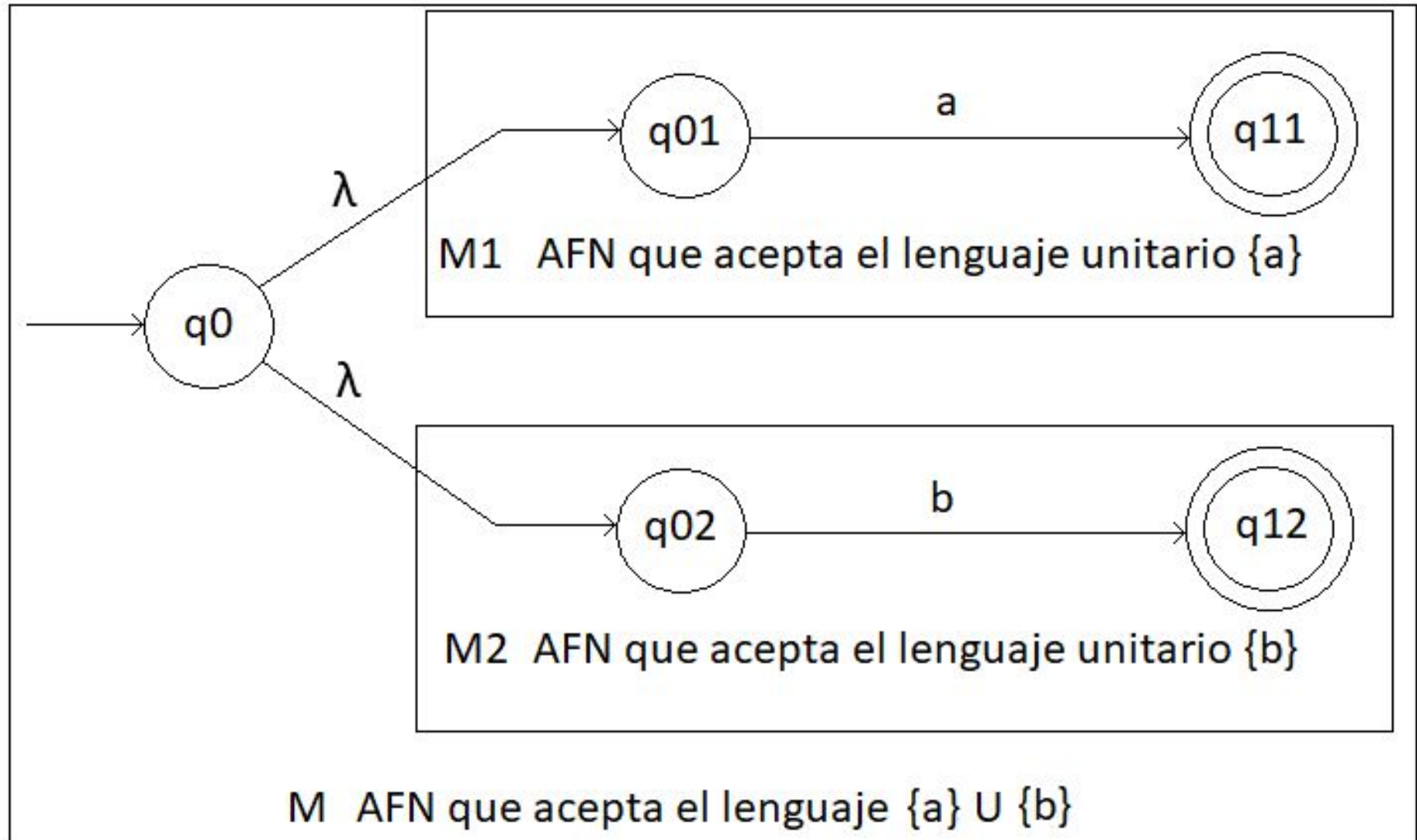
Si $M1 = (Q1, V1, \Delta1, q01, F1)$ y $M2 = (Q2, V2, \Delta2, q02, F2)$ son AFN, podemos unir $M1$ y $M2$ en un nuevo AFN que acepte $L(M1) \cup L(M2)$, añadiendo un nuevo estado inicial $q0$ y dos λ -transiciones, una de $q0$ a $q01$ y otra de $q0$ a $q02$.

La construcción formal de este nuevo AFN $M = (Q, V, \Delta, q0, F)$ viene dado por:

- $V=V1 \cup V2$
- $F=F1 \cup F2$
- $Q=Q1 \cup Q2 \cup \{q0\}$, donde $q0$ es el nuevo estado inicial
- Δ se define de forma que se incluyan todas las transiciones de $\Delta1, \Delta2$ y las dos nuevas λ -transiciones de $q0$ a $q01$ y de $q0$ a $q02$.

AUTÓMATAS FINITOS

Ejemplo:



AUTÓMATAS FINITOS

Autómatas finitos para concatenación de expresiones regulares:

Si $M1 = (Q1, V1, \Delta1, q01, F1)$ y $M2 = (Q2, V2, \Delta2, q02, F2)$ son AFN, podemos unir $M1$ y $M2$ en un nuevo AFN que acepte $L(M1)L(M2)$.

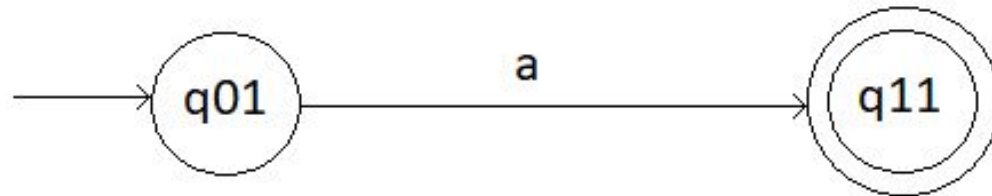
Se necesita un AFN que reconozca una cadena de $L(M1)$ y después reconozca una de $L(M2)$. Esto se realiza de forma no determinista pasando del estado final de $M1$ al estado inicial de $M2$ por medio de una λ -transición.

La construcción formal de este nuevo AFN $M = (Q, V, \Delta, q0, F)$ viene dado por:

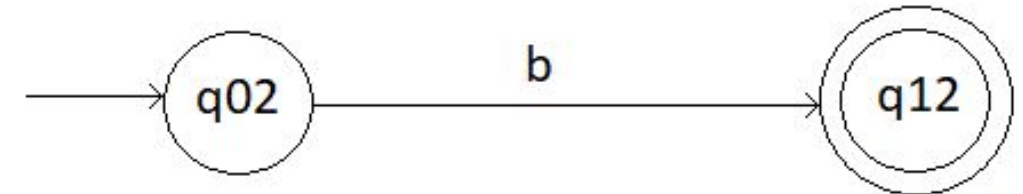
- $V=V1 \cup V2$
- $F=F2$
- $q0=q01$
- $Q=Q1 \cup Q2$
- $\Delta=\Delta1 \cup \Delta2 \cup (\lambda\text{-transiciones de los estados finales de AFN1 al estado inicial de AFN2})$

AUTÓMATAS FINITOS

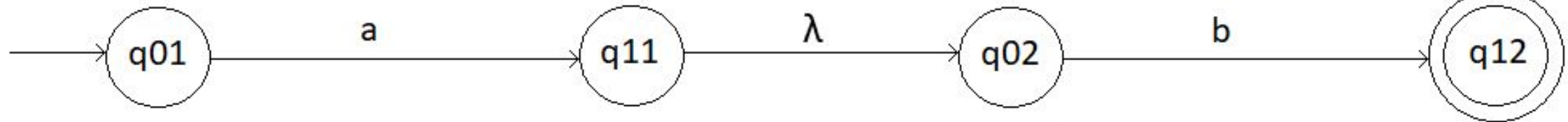
Ejemplo:



M1 AFN que acepta el lenguaje unitario {a}



M2 AFN que acepta el lenguaje unitario {b}



M AFN que acepta el lenguaje {a}{b}

AUTÓMATAS FINITOS

Autómatas finitos para cierre de expresiones regulares:

Se puede deducir un procedimiento para construir una AFN que acepte $L(M)^*$ para el AFN $M = (Q, V, \Delta, q_0, F)$, como sigue:

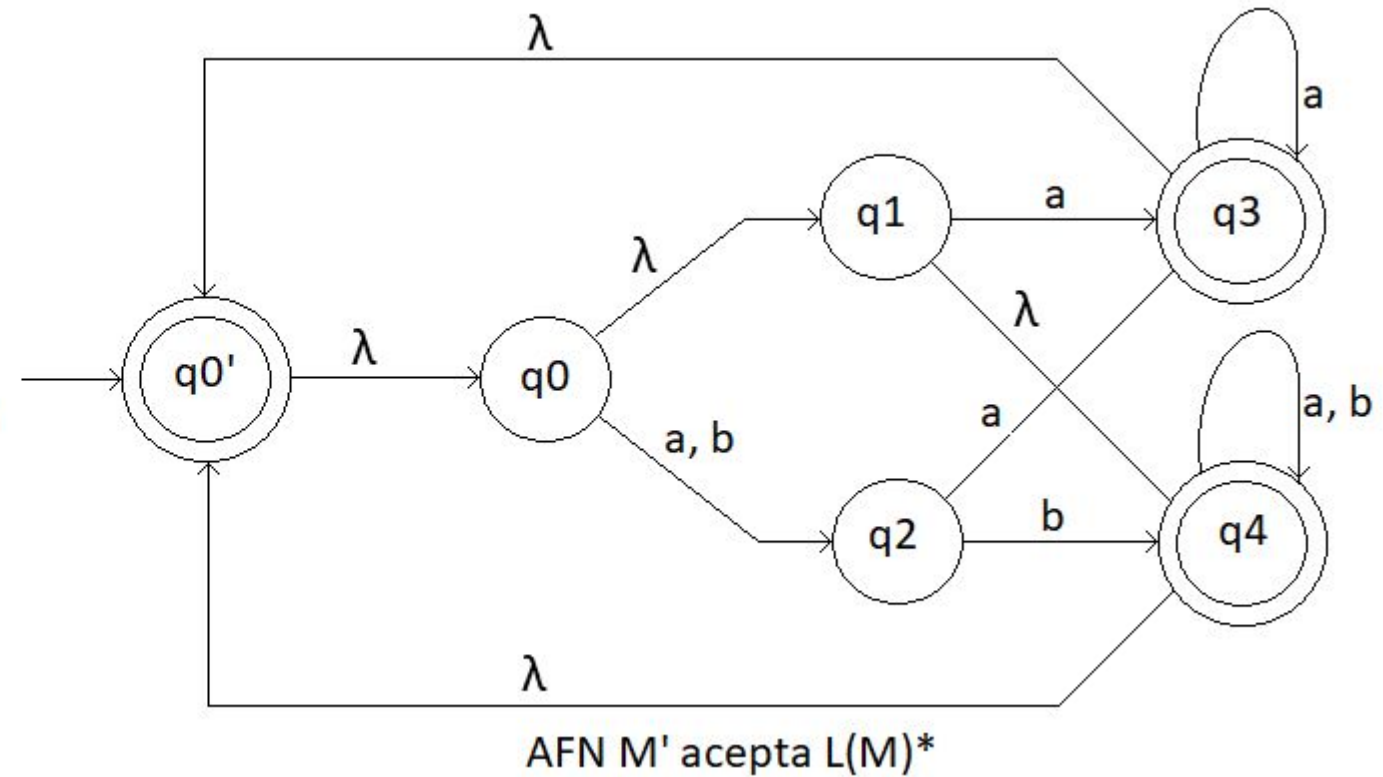
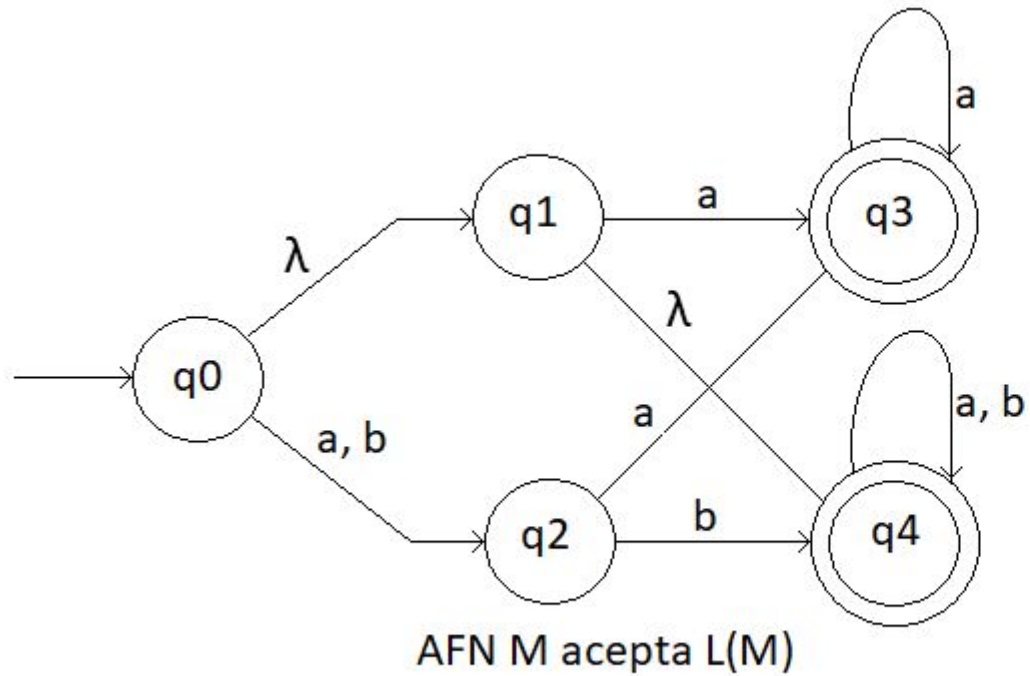
Se añade un nuevo estado inicial q_0' . Este estado será además un estado de aceptación con el fin de que λ sea aceptada; entonces, se permite una λ -transición desde q_0' a el antiguo estado inicial q_0 . Por tanto, M comenzará una vez que M' se encuentre en q_0' . Se tendrá además, una λ -transición desde todos los estados de aceptación hasta el estado inicial q_0' . Una vez que la cadena de $L(M)$ ha sido agotada, el análisis puede continuar a partir del estado inicial de M o terminar en q_0' . El autómata resultante será $M' = (Q', V', \Delta', q_0', F')$, donde:

- $V' = V \cup \lambda$
- $Q' = Q \cup \{q_0'\}$
- $F' = \{q_0'\}$
- $\Delta' = \Delta \cup \{(q_0', \lambda, q_0)\} \cup (F \cup \{\lambda\} \cup \{q_0'\})$

Donde la terna genérica (p, α, q) indica que hay una transición del estado p al estado q a través del carácter α . En este caso, el carácter intermedio será λ en todos los casos. Es decir, Δ' está formado por Δ y todas las λ -transición de un estado final de M a q_0' .

AUTÓMATAS FINITOS

Ejemplo:

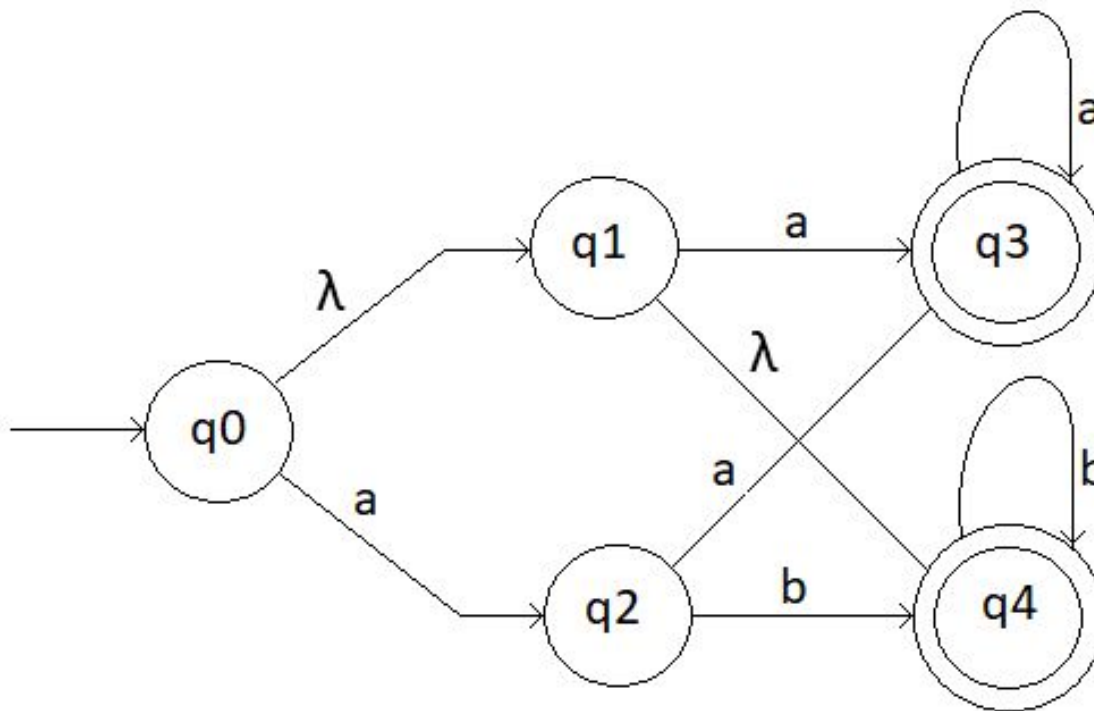


AUTÓMATAS FINITOS

Derivación de expresiones regulares a partir de un AFN:

Sea el AF $M = (Q, V, \Delta, q_0, F)$, para todo estado q_i , se define A_i como el conjunto de todas las cadenas que hacen que M pase de q_i a un estado de aceptación. Por la definición podemos observar que $A_0 = L(M)$.

Ejemplo:



$$A_4 = b^*$$

$$A_3 = a^*$$

$$A_2 = aa^* \cup bb^*$$

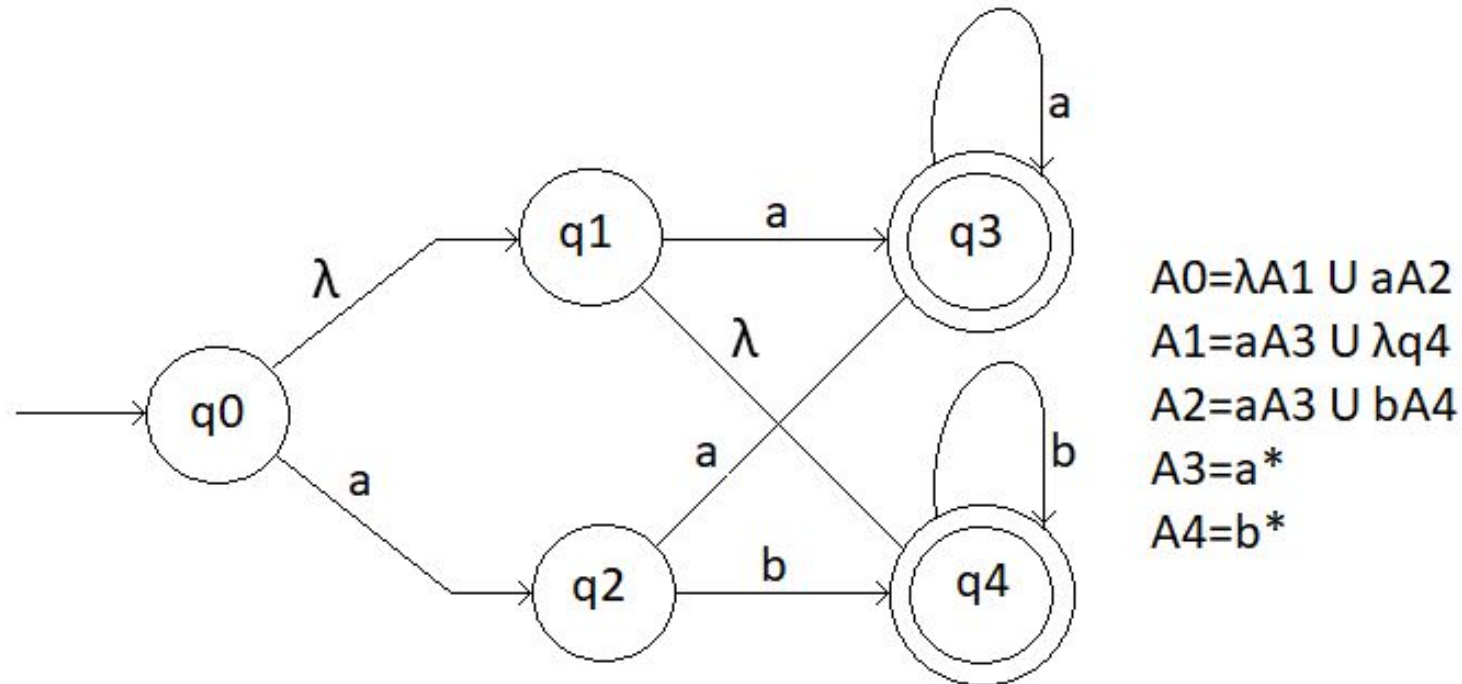
$$A_1 = aa^* \cup b^*$$

$$A_0 = aa^* \cup b^* \cup aaa^* \cup abb^*$$

AUTÓMATAS FINITOS

Sea q_j un estado tal que se puede llegar desde q_i a través de una única transición $\Delta(q_i, \alpha)$. Es decir, $q_i \rightarrow \alpha q_j$. Entonces, A_i contiene a αA_j . Viendo el ejemplo de abajo podemos inferir que A_i esta formada por la unión de todas las cadenas αA_j , donde cada α es el carácter por el que se llega del estado q_i a cada uno de los estado q_j adyacente a q_i .

Ejemplo:



A partir del ejemplo podemos ver que dado un autómata finito M , se puede obtener un sistema de ecuaciones, que si es resuelto (por sustitución), se obtiene una expresión regular para el lenguaje que acepta M .

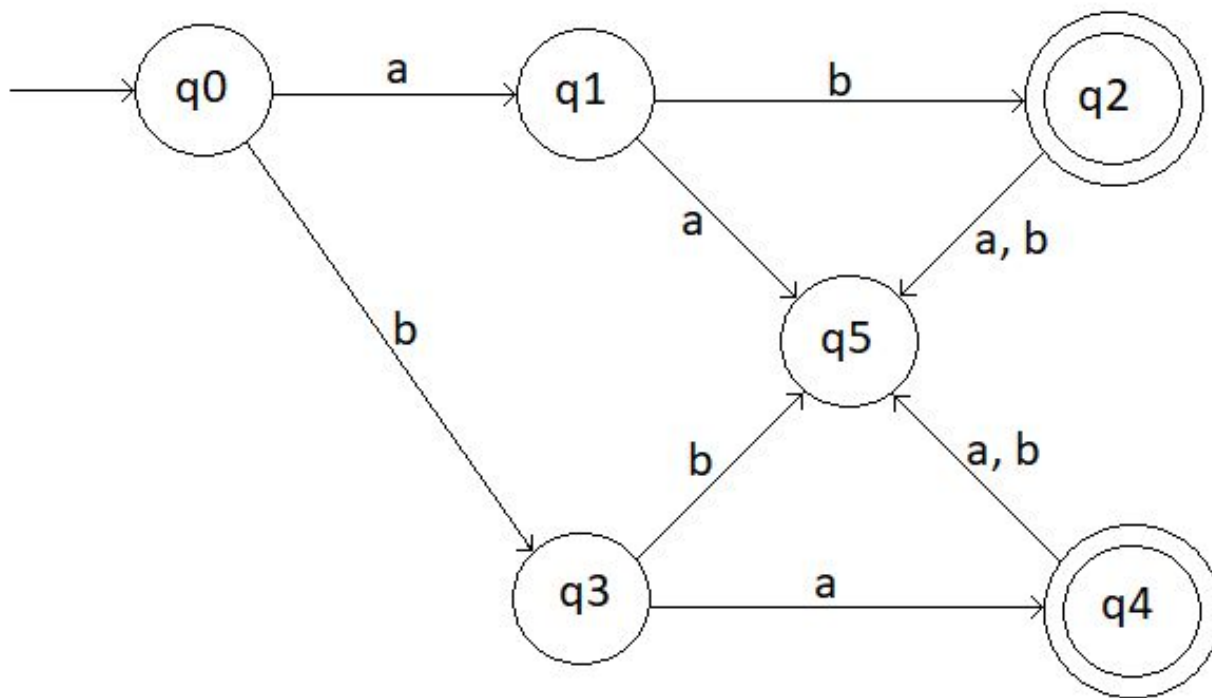
AUTÓMATAS FINITOS

Lema de Arden: Una ecuación de la forma $X = AX \cup B$, donde λ no pertenece a A , tiene una solución única $X = A^*B$.

A partir de este lema y con la construcción de un sistema de ecuaciones de expresiones regulares para un autómata finito M , se puede obtener fácilmente la expresión regular que represente al lenguaje que es aceptado por M , es decir, $L(M)$.

AUTÓMATAS FINITOS

Ejemplo 1:



$$A_0 = aA_1 \cup bA_3$$

$$A_1 = bA_2 \cup aA_5$$

$$A_2 = \lambda \cup aA_5 \cup bA_5$$

$$A_3 = aA_4 \cup bA_5$$

$$A_4 = \lambda \cup aA_5 \cup bA_5$$

$$A_5 = \emptyset$$

Sustituyendo:

$$A_4 = \lambda$$

$$A_3 = a$$

$$A_2 = \lambda$$

$$A_1 = b$$

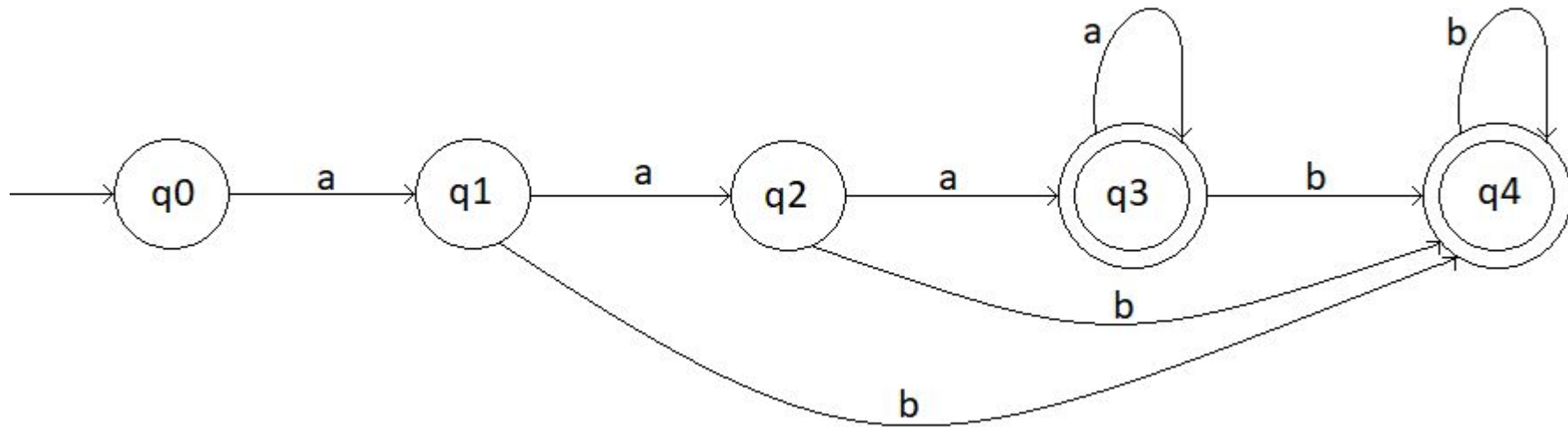
$$A_0 = ab \cup ba$$

Luego:

$$L(M) = ab \cup ba$$

AUTÓMATAS FINITOS

Ejemplo 2:



$$A_0 = aA_1$$

$$A_1 = aA_2 \cup bA_4$$

$$A_2 = aA_3 \cup bA_4$$

$$A_3 = \lambda \cup aA_3 \cup bA_4$$

$$A_4 = \lambda \cup bA_4$$

$$\text{Usando } X = AX \cup B \Rightarrow X = A^*B$$

$$A_4 = bA_4 \cup \lambda \Rightarrow A_4 = b^*\lambda = b^*$$

$$A_3 = aA_3 \cup bA_4 \cup \lambda = aA_3 \cup bb^* \cup \lambda = aA_3 \cup b^+ \cup \lambda \Rightarrow A_3 = aA_3 \cup b^* = a^*b^*$$

$$A_2 = aa^*b^* \cup bb^* = a^+b^* \cup b^+$$

$$A_1 = a(a^+b^* \cup b^+) \cup bb^* = aa^+b^* \cup ab^+ \cup b^+$$

$$A_0 = aA_1 = a^2a^+b^* \cup a^2b^+ \cup ab^+ \Rightarrow L(M) = a^2a^+b^* \cup a^2b^+ \cup ab^+$$

AUTÓMATAS FINITOS

Autómatas finitos y gramáticas regulares:

Los autómatas finitos son mecanismos reconocedores de lenguajes y las gramáticas regulares son mecanismos generadores y ambos tratan con la misma clases de lenguajes: los lenguajes regulares.

Teorema (AF $\rightarrow$ GR): Si L es un lenguaje aceptado por una autómatata finito M , entonces existe una gramática regular G tal que $L = L(M) = L(G)$.

Teorema (GR $\rightarrow$ AF): Si L es un lenguaje generado por una gramática regular G , entonces existe un autómatata finito M tal que $L = L(G) = L(M)$.

AUTÓMATAS FINITOS

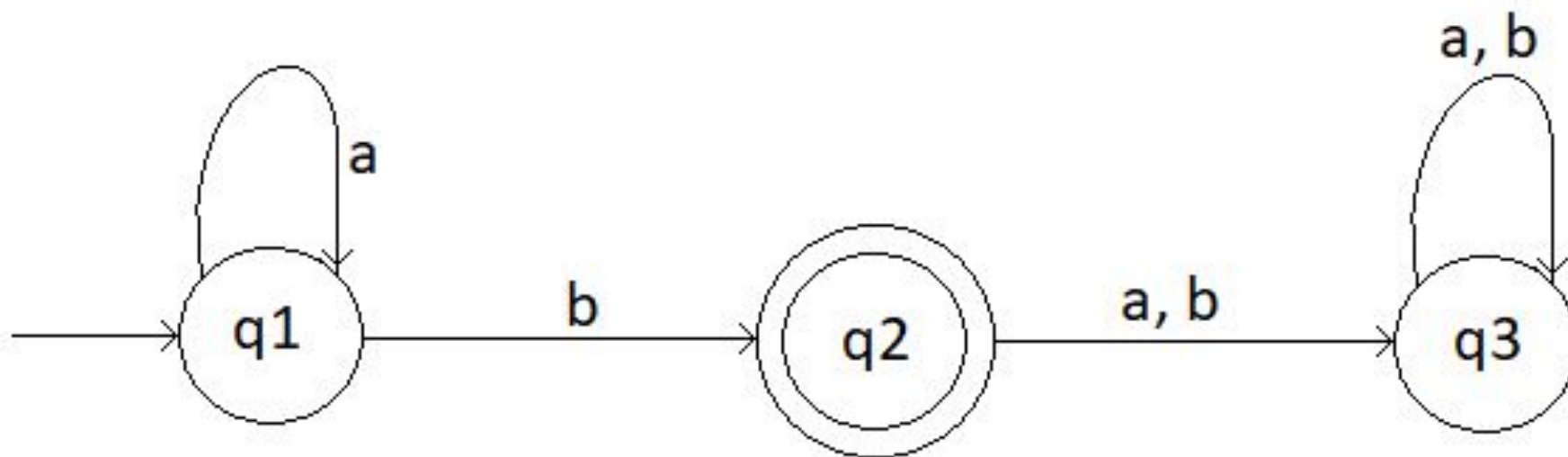
Obtención de una gramática regular a partir de un autómata finito :

Sea L un lenguaje regular. Se puede obtener una gramática regular que genere L por medio de un AFD $M = (Q, V, \delta, q_0, F)$ para el cual $L=L(M)$. Se define $G = (V_N, V_T, S, P)$ por:

- $V_N = Q$
- $V_T = V$
- $S = q_0$
- P contiene todas las reglas (q, ap) donde $\delta(q, a) = p$
- P contiene la regla $q \rightarrow \lambda$ si q es un estado de aceptación del AFD

AUTÓMATAS FINITOS

Ejemplo:



Producciones de G:

$q1 \rightarrow aq1 \mid bq2$

$q2 \rightarrow aq3 \mid bq3$

$q3 \rightarrow aq3 \mid bq3$

Ecuaciones de M (AFD):

$A1 = aA1 \cup bA2$

$A2 = \lambda \cup aA3 \cup bA3$

$A3 = aA3 \cup bA3 = \emptyset \Rightarrow A2 = \lambda$

$A1 = aA1 \cup b\lambda \Rightarrow A1 = a^*b$

Lenguaje $L(M)=L(G)$:

$L(M)=L(G)=a^*b$

AUTÓMATAS FINITOS

Obtención de un autómata finito a partir de una gramática regular:

Es posible partir de una gramática regular G y construir un AFN M de forma que $L(G) = L(M)$. Sea $G = (V_N, V_T, S, P)$ una gramática regular; se define $M = (Q, V, \Delta, q_0, F)$ mediante:

- $Q = V_N \cup \{f\}$, donde f es un símbolo nuevo
- $q_0 = S$
- $F = \{f\}$
- Δ se construye como se indica a continuación, a partir de las producciones de P :
 1. Si $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n B$ es una producción de P con A y B como no terminales, entonces se añadirán a Q los nuevos estados $q_1 q_2 \dots q_{n-1}$ y las transformaciones siguientes:
$$\Delta(A, \alpha_1) = q_1; \Delta(q_1, \alpha_2) = q_2; \Delta(q_2, \alpha_3) = q_3; \dots \Delta(q_{n-1}, \alpha_n) = B$$
 2. Si $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n$ es una producción de P , entonces se añadirán a Q los nuevos estados $q_1 q_2 \dots q_{n-1}$ y a Δ , las transiciones siguientes:
$$\Delta(A, \alpha_1) = q_1; \Delta(q_1, \alpha_2) = q_2; \Delta(q_2, \alpha_3) = q_3; \dots \Delta(q_{n-1}, \alpha_n) = f$$

AUTÓMATAS FINITOS

Ejemplo:

$S \rightarrow aB \mid bA \mid \lambda$

$A \rightarrow abaS$

$B \rightarrow babS$

$S \rightarrow aB \Rightarrow \Delta(S, a) = B$

$S \rightarrow bA \Rightarrow \Delta(S, b) = A$

$S \rightarrow \lambda \Rightarrow \Delta(S, \lambda) = q5$

$A \rightarrow abaS$ genera

$\Delta(A, a) = q1$

$\Delta(q1, b) = q2$

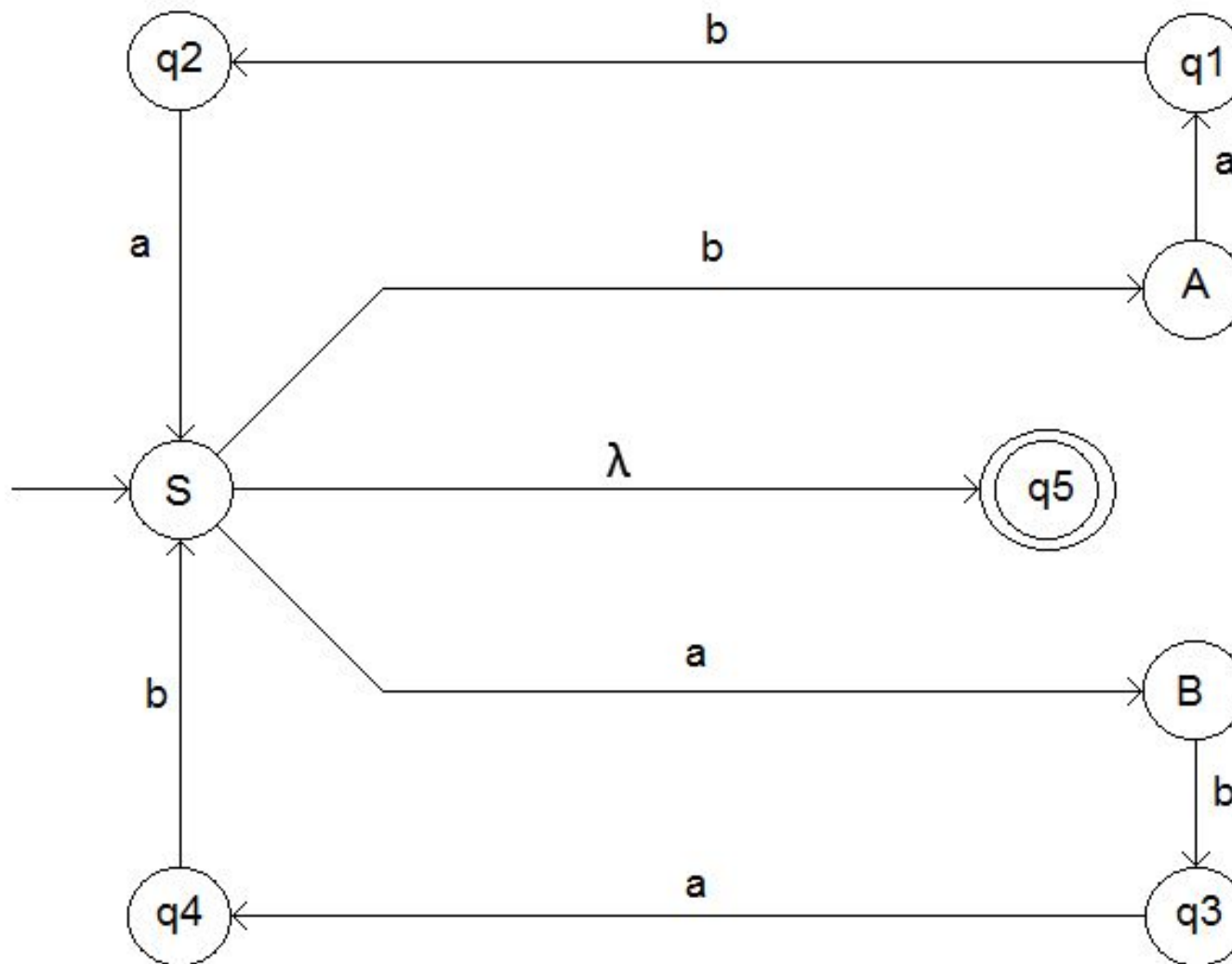
$\Delta(q2, a) = S$

$B \rightarrow babS$ genera

$\Delta(B, b) = q3$

$\Delta(q3, a) = q4$

$\Delta(q4, b) = S$



AUTÓMATAS FINITOS

Equivalencia entre autómatas finitos:

Dos autómatas finitos M y M' son equivalentes si y sólo si aceptan el mismo lenguaje, esto es, $L(M) = L(M')$.

Teorema: Un lenguaje es aceptado por un AFND si y sólo si es aceptado por un AFD.

Teorema: Un lenguaje es aceptado por un AFND- λ si y sólo si es aceptado por un AFND.

AUTÓMATAS FINITOS

Algoritmo de paso de AFND a AFD:

Entrada: Un AFND $M_N = (Q, V, \Delta, q_0, F)$

Salida: Un AFD $M_D = (Q', V, \delta, q'_0, F')$ tal que $L(M_D) = L(M_N)$

Algoritmo:

- 1) Se parte del estado inicial q_0 y se asigna al estado inicial q'_0 .
- 2) El estado q'_0 está compuesto por q_0 y todos los estados q_i a los que se puede llegar desde q_0 con λ -transiciones: $q'_0 = \{q_0, q_1, q_2, \dots, q_i, \dots, q_n\}$ donde $\Delta(q_0, \lambda) = q_i$. Se agrega q'_0 a Q' .
- 3) Se toma el primer símbolo del alfabeto a , y se arma el estado q'_1 con todos los estados q_j de Q a lo que se llega desde cada uno de los estados q_i que constituyen q'_0 al leer a . Se agrega q'_1 los estados a lo que se llega con λ -transiciones. Se agrega q'_1 a Q' .
- 4) Se repite el punto 3 para el resto de los símbolos del alfabeto.
- 5) Se marca q'_0 dentro de Q' para indicar que ya fue analizada.
- 6) Se toma el siguiente estado sin marcar que se encuentra en Q' .
- 7) Se repite para este nuevo estado los pasos de 3 a 5.
- 8) Se repiten los pasos anteriores hasta que no haya estados sin marcar en Q' .
- 9) F' quedará constituido por todos los estados de Q' que contengan algún estado final q_f de F .

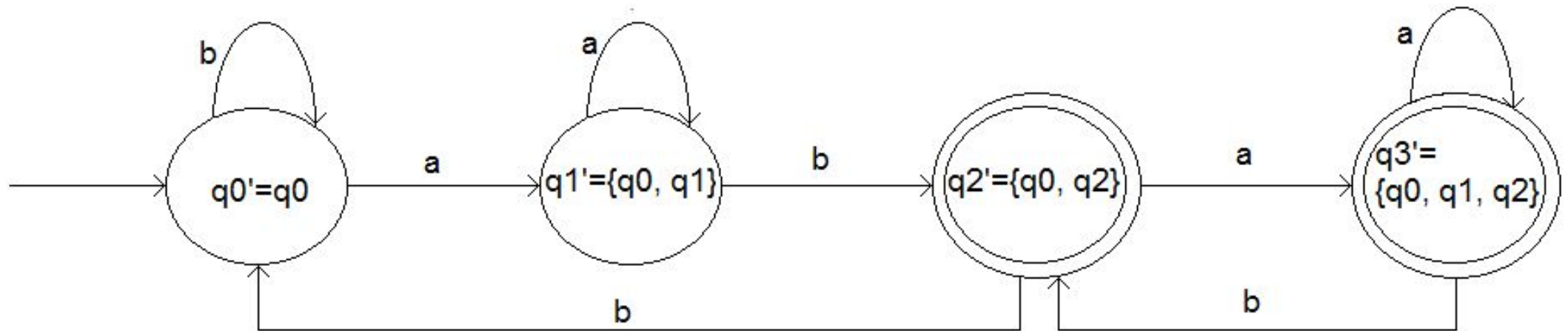
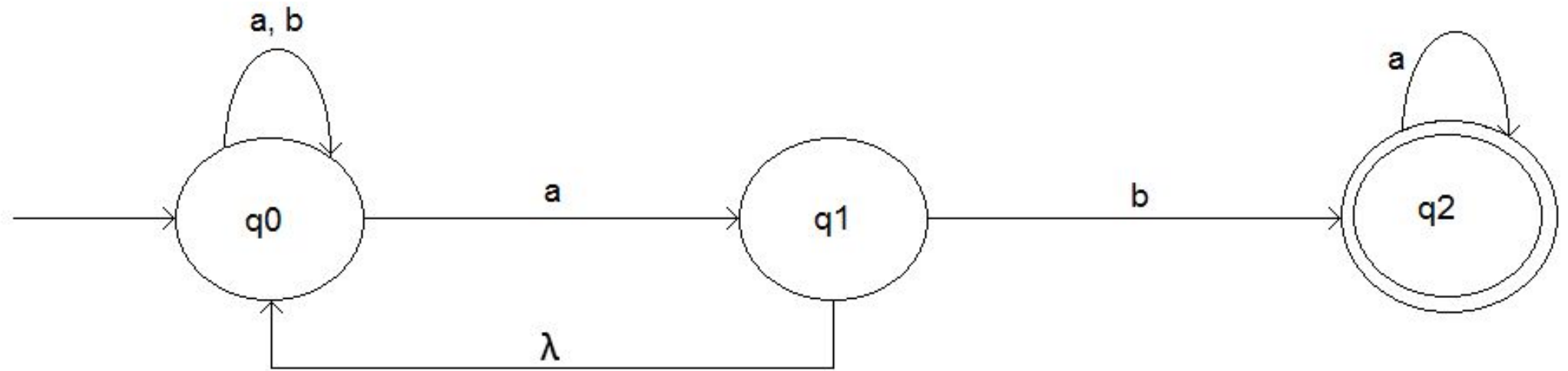
AUTÓMATAS FINITOS

Ejemplo:

Δ	a	b	λ
$\rightarrow q_0$	q_0, q_1	q_0	$\emptyset$
q_1	$\emptyset$	q_2	q_0
$\#q_2$	q_2	$\emptyset$	$\emptyset$

δ	a	B	Q' marcada
$\rightarrow q_0' = q_0$	$\{q_0, q_1\} = q_1'$	$\{q_0\} = q_0'$	$q_0' = \{q_0\}$
q_1'	$\{q_0, q_1\} = q_1'$	$\{q_0, q_2\} = q_2'$	$q_1' = \{q_0, q_1\}$
$\#q_2'$	$\{q_0, q_1, q_2\} = q_3'$	$\{q_0\} = q_0'$	$q_2' = \{q_0, q_2\}$
$\#q_3'$	$\{q_0, q_1, q_2\} = q_3'$	$\{q_0, q_2\} = q_2'$	$q_3' = \{q_0, q_1, q_2\}$

AUTÓMATAS FINITOS



AUTÓMATAS FINITOS

Minimización de un AFD:

Se puede obtener un AFD equivalente a uno dado que tenga el menor número de estados posibles. Para ello es necesario definir antes una relación de equivalencia en el conjunto de estados de un AFD.

Definición: Un estado q de un AFD es accesible si $\exists x \in V^*$ tal que $(q_0, x) \vdash^* (q, \lambda)$. En otro caso el estado es inaccesible.

Definición: Decimos que dos estados p y q de un AFD son equivalentes y lo notamos $p \approx q$, si cumplen:
 $\forall w \in V^* : \text{si } [(q, w) \vdash^* (q', \lambda) \wedge (p, w) \vdash^* (p', \lambda)] \text{ entonces } [q' \in F \Leftrightarrow p' \in F]$

Por otro lado, p y q son distinguibles si no son equivalentes, o lo que es lo mismo, existe una cadena w tal que se produce una de las dos condiciones siguientes:

$$\begin{aligned} q' \in F \text{ y } p' \notin F \\ q' \notin F \text{ y } p' \in F \end{aligned}$$

AUTÓMATAS FINITOS

Observaciones:

1. Si tenemos dos estados $q \in F$ y $p \notin F$ entonces podemos asegurar que son distinguibles ya que tomando $w = \lambda$ se cumple la definición.
2. Si sabemos que dos estados (p, q) son distinguibles y se tiene que $pa = \delta(p, a)$, $qa = \delta(q, a)$ entonces podemos asegurar que (p, q) son distinguibles, ya que si una cadena w hace distinguibles a (pa, qa) entonces la cadena aw hace distinguibles a (p, q) .
3. $\approx$ define una relación de equivalencia en el conjunto de estados del autómata y una forma de reducir el número de estados de un AFD será encontrar las clases de equivalencia en el conjunto de estados y a partir de ahí construir un autómata cuyos estados sean las clases de equivalencia.

AUTÓMATAS FINITOS

Definición: Dado un AFD $M = (Q, V, \delta, q_0, F)$ se define el autómata cociente de M como $M \approx = (Q', V, \delta', q'_0, F')$ donde:

$$Q' = Q / \approx$$

$$\delta'([q], a) = [\delta(q, a)]$$

$$q'_0 = [q_0]$$

$$F' = F / \approx$$

Teorema: Dado un autómata finito determinista M , el autómata cociente $M \approx$ es el autómata mínimo equivalente a M . Este autómata mínimo es único, salvo isomorfismos (renombramiento de estados).

AUTÓMATAS FINITOS

Algoritmo de Minimización de un AFD:

Entrada: Un AFD $M = (Q, V, \delta, q_0, F)$ con $Q = \{q_0, \dots, q_n\}$, $V = \{a_1, \dots, a_m\}$

Salida: AFD mínimo $M \approx = (Q', V, \delta', q'_0, F')$

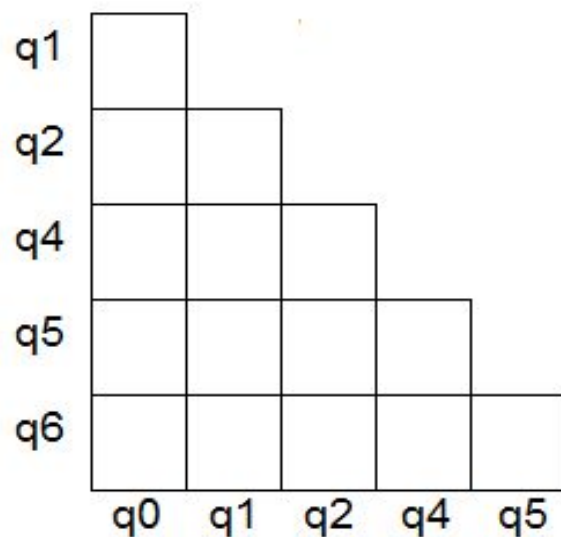
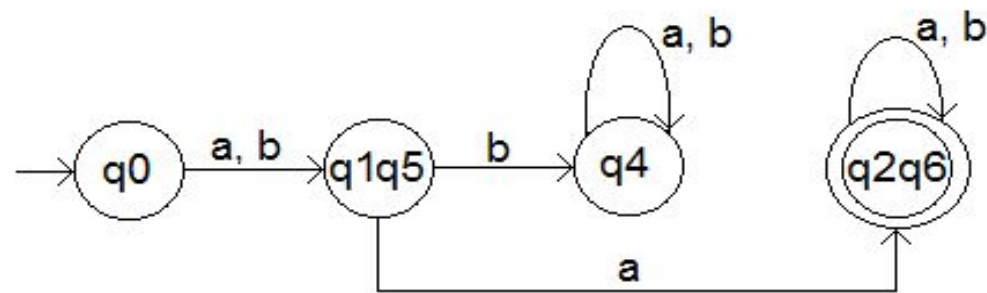
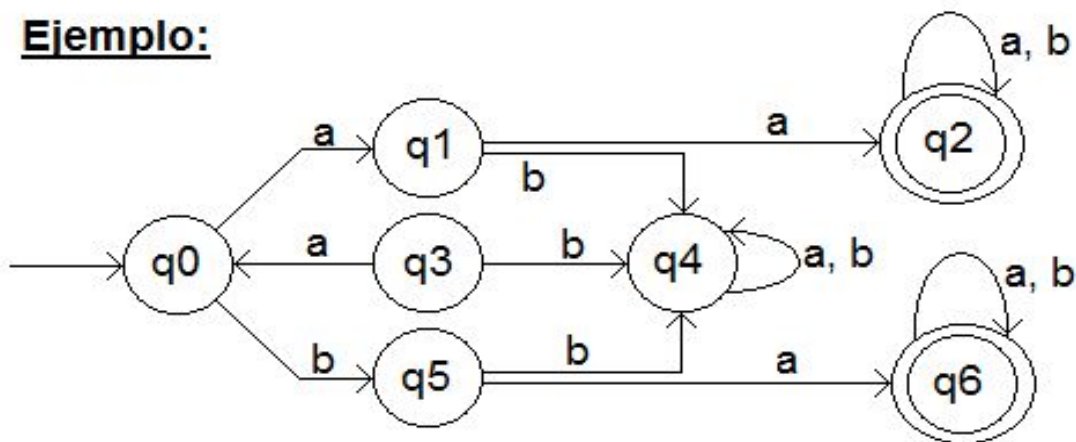
Algoritmo:

- 1) Eliminar los estados inaccesibles.
- 2) Construir una tabla con todos los pares (p, q) de estados restantes.
- 3) Marcar en la tabla aquellas entradas donde un estado es final y el otro estado es no-final, es decir, pares de estados claramente distinguibles.
- 4) Para cada par (p, q) y cada símbolo del alfabeto a , tal que $r = \delta(p, a)$ y $s = \delta(q, a)$:
 - a) Si (r, s) ya ha sido marcado, entonces p y q también son distinguibles, por lo tanto debe marcarse (p, q) .
 - b) De lo contrario, colocar (p, q) en una lista asociada a la entrada (r, s) .
- 5) Agrupar los pares de estado no marcados.

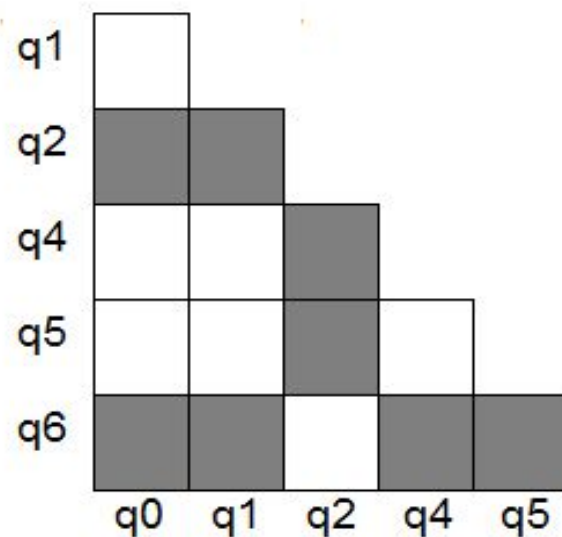
NOTA: si luego del tercer paso, la tabla queda completamente marcada, entonces el AFD ya era mínimo. El paso 4 se repite hasta que no puedan marcarse estados.

AUTÓMATAS FINITOS

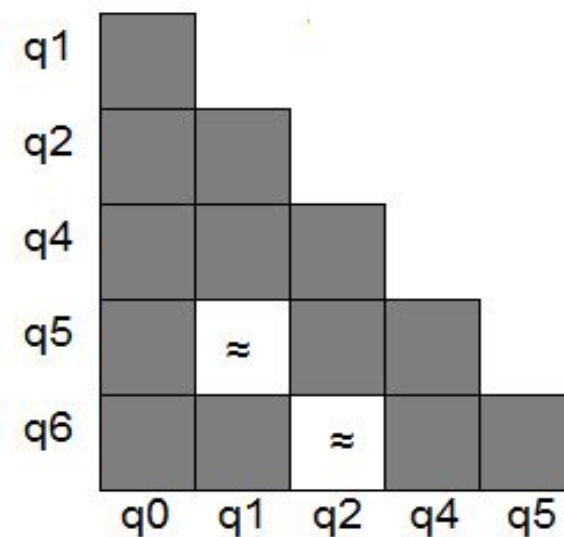
Ejemplo:



paso 1



paso 2



paso 3

$q1 \approx q5$

$q2 \approx q6$

paso 4

AUTÓMATAS FINITOS

Aplicaciones: análisis léxico

Una de las aplicaciones más importantes de los AFs es la construcción de analizadores léxicos.

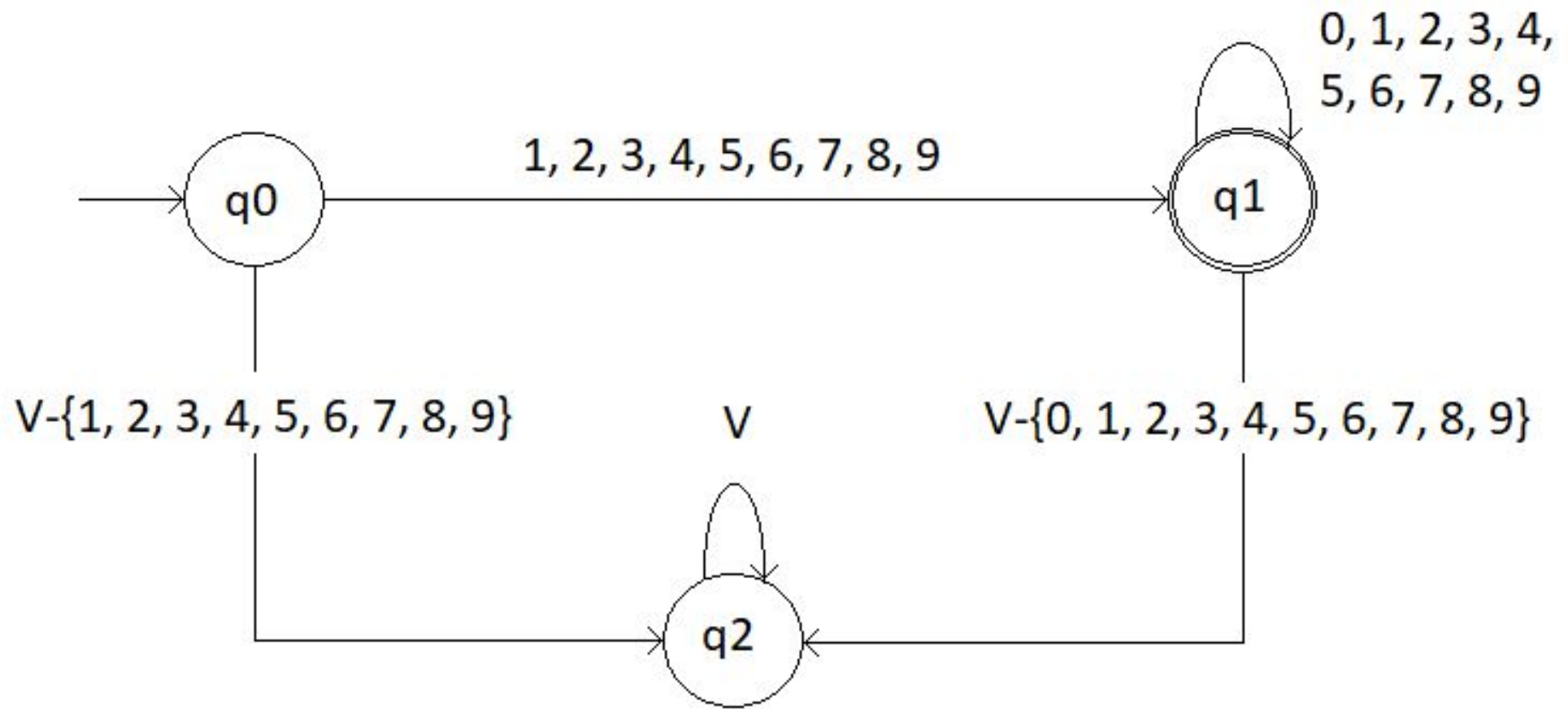
Un problema común en la programación de computadoras es el de tener la seguridad de que los datos de entrada de un programa son correctos. Por ejemplo, si se espera un entero sin signo como dato de entrada y el usuario confunde uno de los dígitos con un carácter no numérico, se puede dar todo tipo de resultados impropios, desde una terminación anormal hasta el cálculo de resultados incorrectos.

La programación cuidadosa pretende construir programas donde se incluyan rutinas de entrada que analicen la información introducida por el usuario y, de alguna forma, prevenir que se aplique información incorrecta al programa.

Si pudiéramos construir un autómata finito que aceptara solamente las cadenas que representan información correcta, entonces tendríamos un modelo para dicha rutina de entrada. Puesto que los autómatas finitos se corresponden con las expresiones regulares, el problema se reduce a especificar la información correcta por medio de expresiones regulares.

AUTÓMATAS FINITOS

En el caso de que la entrada esté formada por enteros sin signo, el lenguaje vendrá dado por $I = \{1, 2, 3, 4, 5, 6, 7, 8, 9\} \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}^*$. Es fácil construir un autómata finito que acepte I :



AUTÓMATAS FINITOS

También es sencillo traducir el autómata finito a un código en un lenguaje de programación; sólo se necesita seguir el rastro de la posición actual en la cadena y del estado actual. A la vez que se recorre la cadena, se cambia de estado según corresponda y, cuando se acaba la cadena, se comprueba a qué estado se ha llegado y, según eso, se acepta o se rechaza la cadena.

Las expresiones regulares se pueden usar para especificar las unidades léxicas presentes en un lenguaje de programación. Los autómatas finitos asociados se usan para reconocer dichas unidades (llamadas componentes léxicos). Dicho análisis es una etapa importante en la compilación de programas.

AUTÓMATAS FINITOS

Ejercicio 1: Sea M un autómata con el siguiente conjunto de entrada $A = \{a, b\}$, conjunto de estados $S = \{s_0, s_1, s_2\}$ con estado inicial s_0 y conjunto de estados de aceptación $Y = \{s_2\}$. Suponga que la función de estado siguiente F de M está dada por la siguiente tabla:

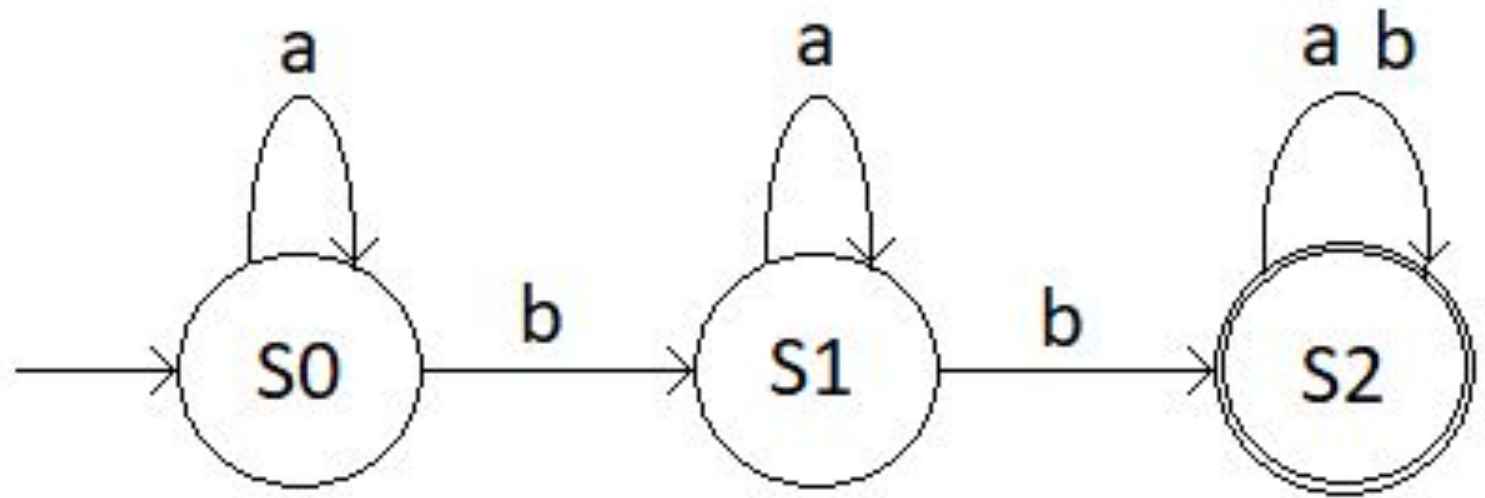
F	a	b
s_0	s_0	s_1
s_1	s_1	s_2
s_2	s_2	s_2

[a] Dibuje el diagrama de estados de M .

[b] Describa el lenguaje aceptado por M .

AUTÓMATAS FINITOS

F	a	b
S_0	S_0	S_1
S_1	S_1	S_2
S_2	S_2	S_2



$$L(M) = a^*ba^*b(a^*b^*)^*$$

AUTÓMATAS FINITOS

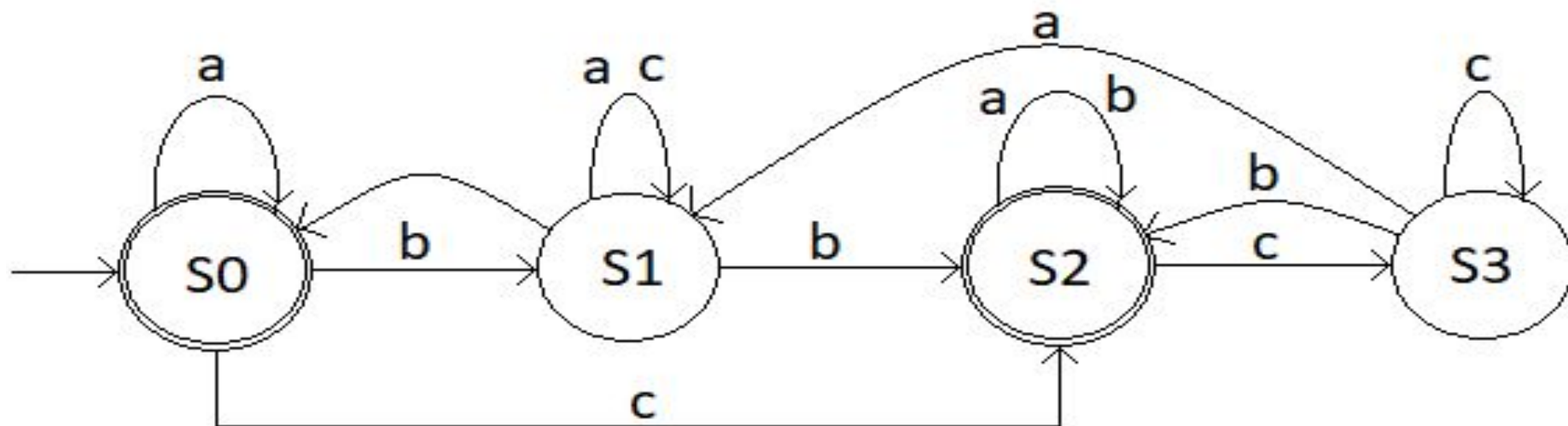
Ejercicio 2: Sea M un autómata con el siguiente conjunto de entrada $A = \{a, b, c\}$, conjunto de estados $S = \{s_0, s_1, s_2, s_3\}$ con estado inicial s_0 y conjunto de estados de aceptación $Y = \{s_0, s_2\}$. Suponga que la función de estado siguiente F de M está dada por la siguiente tabla:

F	a	b	c
s_0	s_0	s_1	s_2
s_1	s_1	s_2	s_1
s_2	s_2	s_2	s_3
s_3	s_1	s_2	s_3

[a] Dibuje el diagrama de estados de M .

[b] Determine si las palabras $w_1 = abcccb$ y $w_2 = abcabcc$ pertenecen al lenguaje aceptado por M .

AUTÓMATAS FINITOS



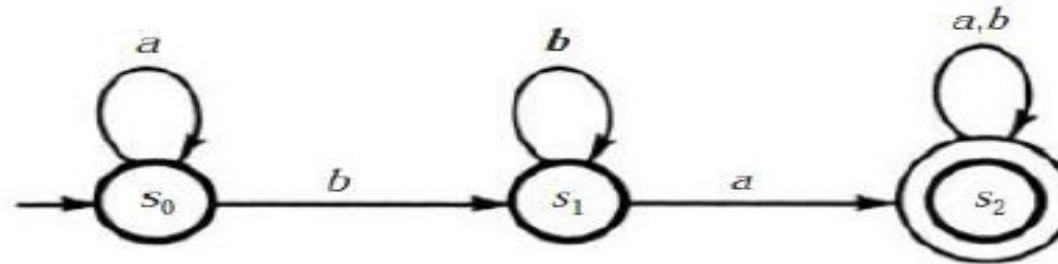
	F	a	b	c
→ #	S_0	S_0	S_1	S_2
	S_1	S_1	S_2	S_1
#	S_2	S_2	S_2	S_3
	S_3	S_1	S_2	S_3

w1=abcccb	w2=abcabcc
$S_0 > S_1 > S_1 >$ $> S_1 > S_1 > S_2$ es aceptada	$S_0 > S_1 > S_1 >$ $S_1 > S_2 > S_3 >$ $> S_3$ no es aceptada

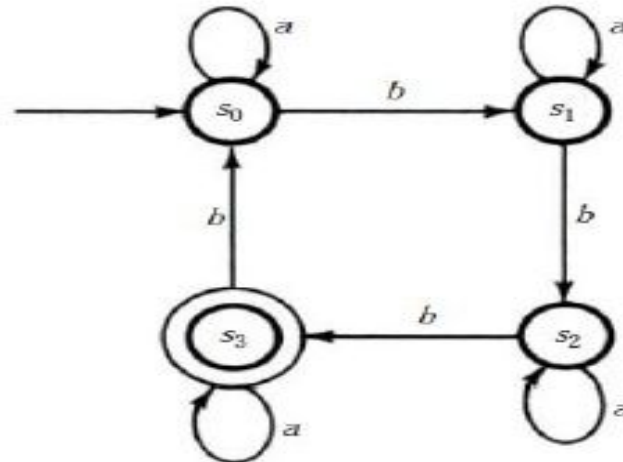
AUTÓMATAS FINITOS

Ejercicio 3: Describa las palabras w que pertenecen al lenguaje L aceptado por los autómatas de las figuras 3-a y 3-b (en lenguaje natural, como en el ejercicio 1-b). Además, genere las tablas correspondientes a sus funciones de estado siguiente.

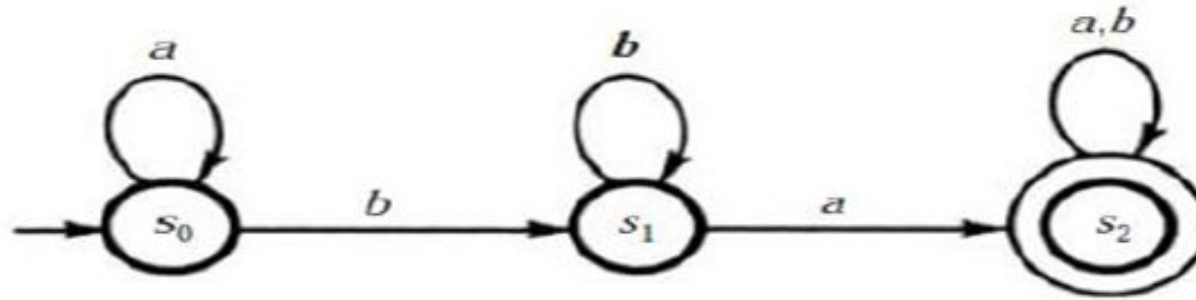
[a]



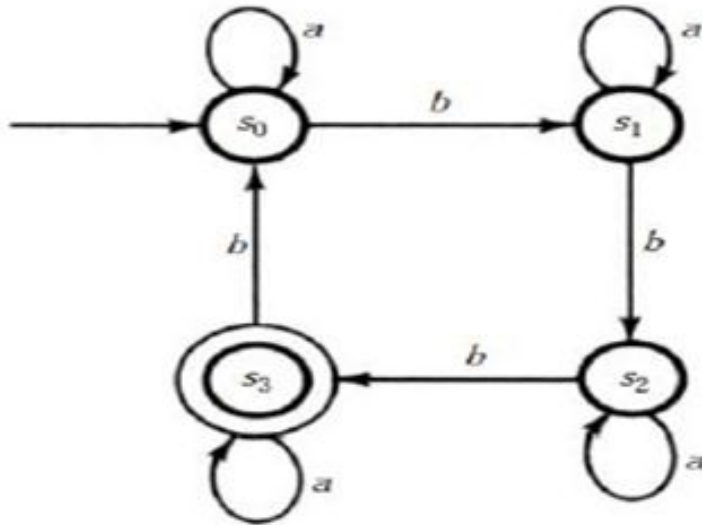
[b]



AUTÓMATAS FINITOS



$$L(M) = a^*bb^*a(a^*b^*)^*$$



$$L(M) = a^*ba^*ba^*ba^*(ba^*ba^*ba^*ba^*)^* = a^*(ba^*)^3((ba^*)^4)^*$$

AUTÓMATAS FINITOS

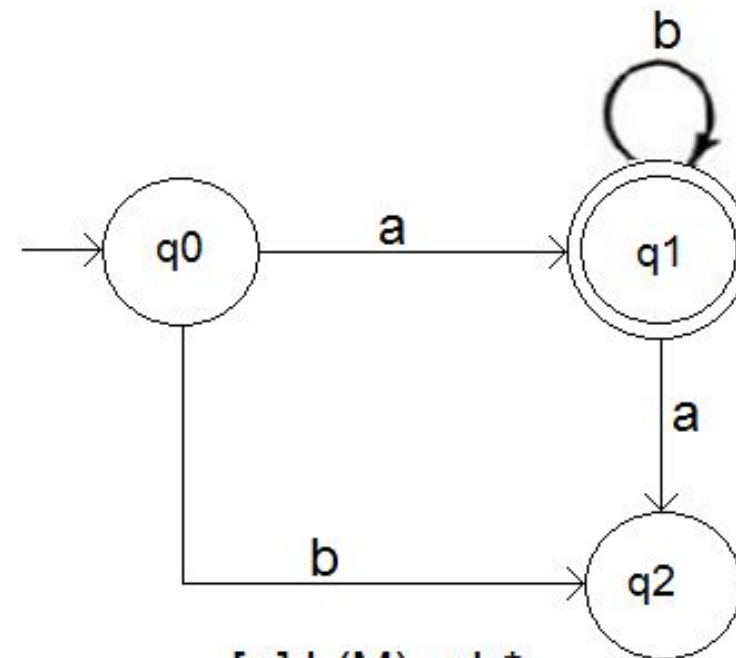
Ejercicio 4: Sea $A = \{a, b\}$. Construya un autómata M tal que $L(M)$ consta de las palabras w donde:

- [a] Aquellas palabras de A que empiecen con una a seguida por (cero o más) b .
- [b] El número de b es divisible entre 3.
- [c] w empieza en a y termina en b .

[a] $L(M)=ab^*$

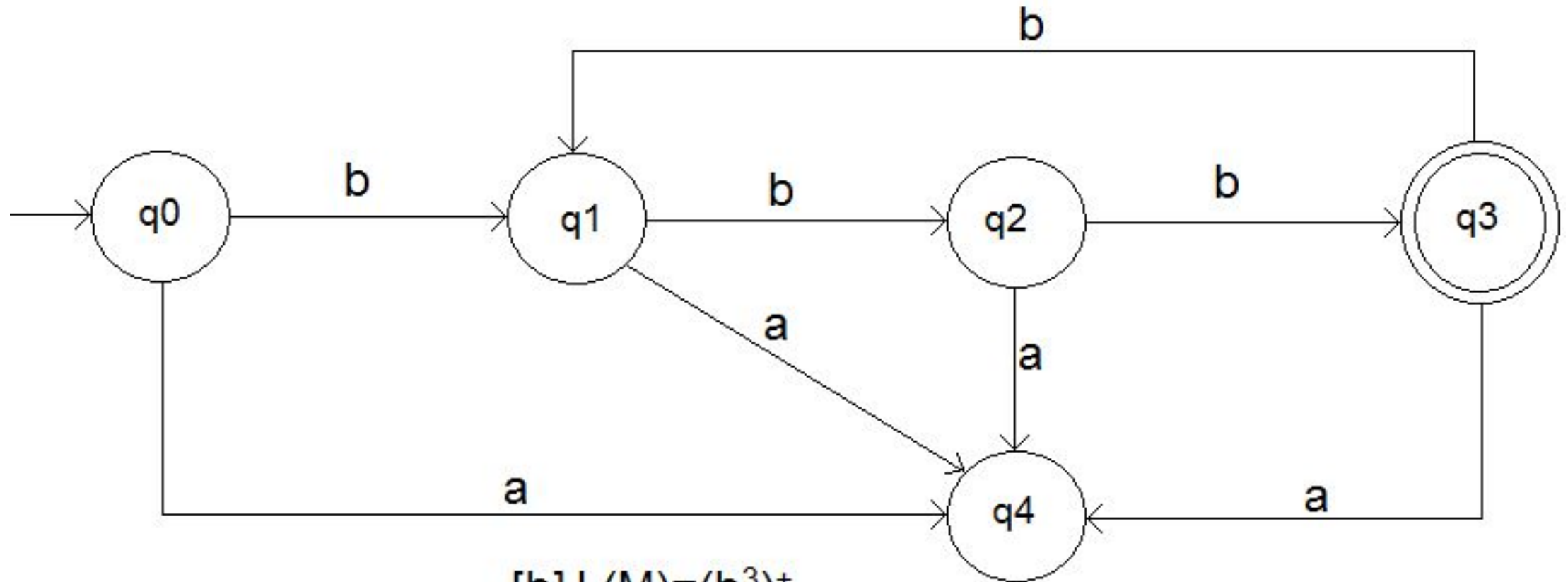
[b] $L(M)=(b^3)^+$

[c] $L(M)=a(a^*b^*)^*b$



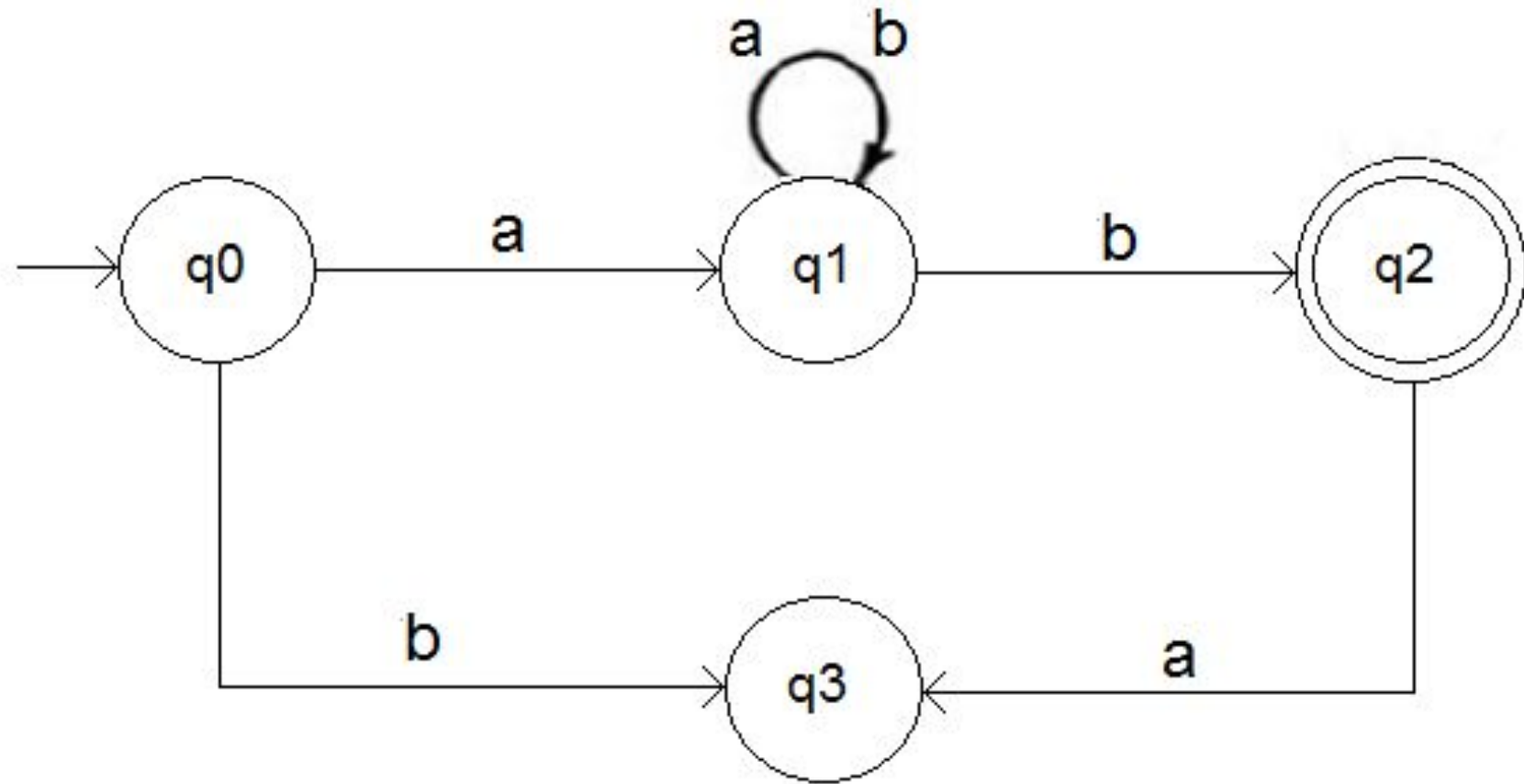
[a] $L(M)=ab^*$

AUTÓMATAS FINITOS



[b] $L(M) = (b^3)^+$

AUTÓMATAS FINITOS



[c] $L(M) = a(a^*b^*)^*b$

AUTÓMATAS FINITOS

Ejercicio 5: Sea $A = \{a, b\}$. Construya un autómata M que acepte el lenguaje:

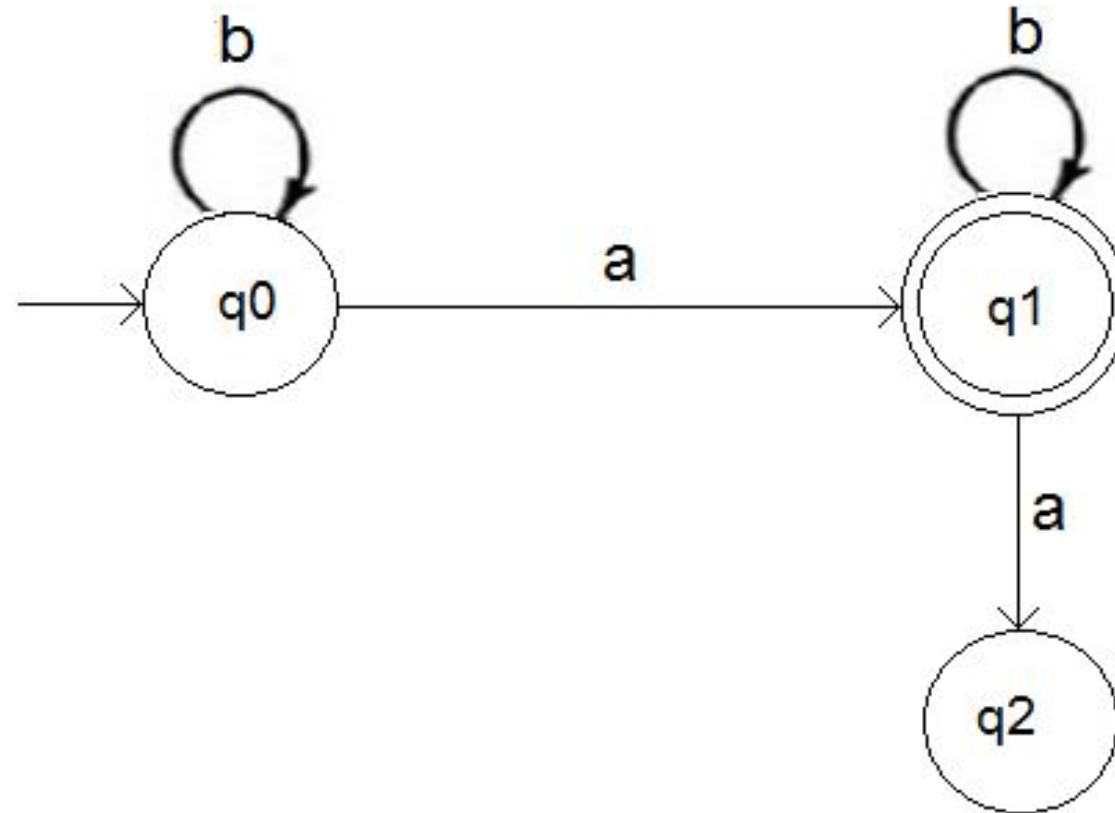
$$[a] L(M) = \{b^r a b^s \mid r > 0, s > 0\}$$

$$[b] L(M) = \{a^r b^s \mid r > 0, s > 0\}$$

$$[a] L(M) = b^* a b^*$$

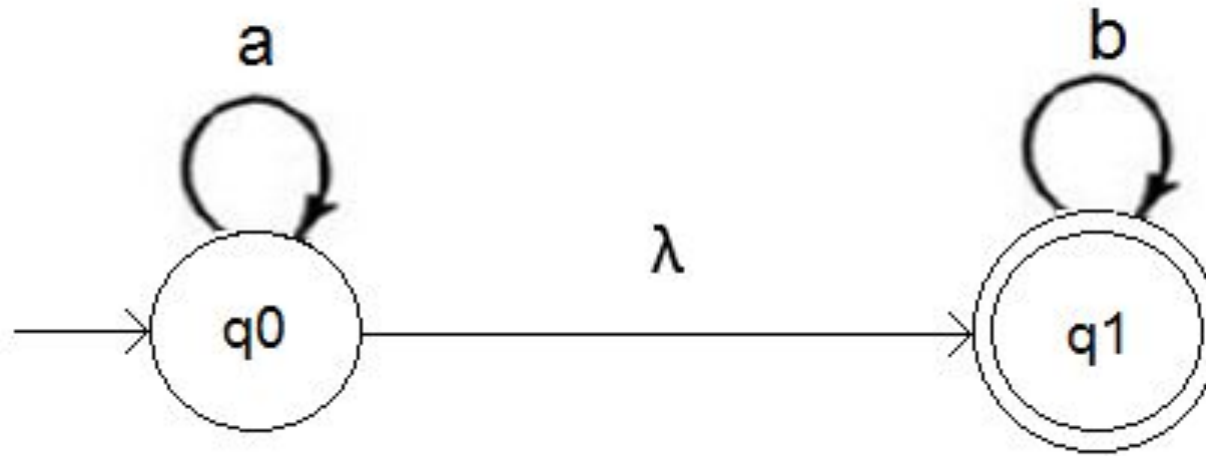
$$[b] L(M) = a^* b^*$$

AUTÓMATAS FINITOS



$$[a] L(M) = \{b^r a b^s \mid r > 0, s > 0\}$$

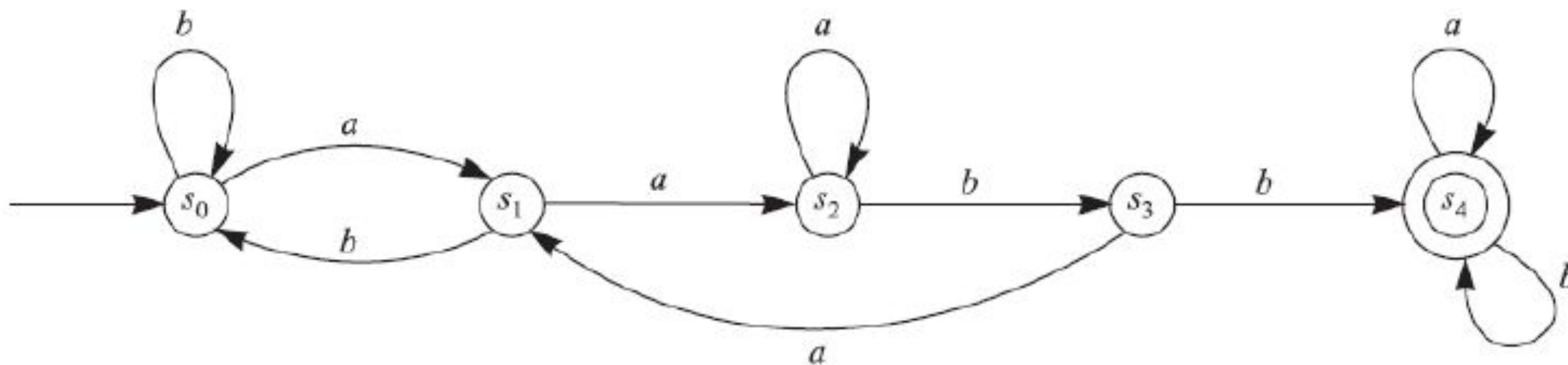
AUTÓMATAS FINITOS



$$[b] L(M) = \{a^r b^s \mid r > 0, s > 0\}$$

AUTÓMATAS FINITOS

Ejercicio 6: Encuentre el lenguaje $L(M)$ aceptado por el autómata M en la siguiente figura:



$$L(M) = b^*a(bb^*a)^*aa^*b(aaa^*b)^*b(a^*b^*)^*$$